

Assignment - Module 3 & 4

Meenu S

February 27, 2026

Introduction

Linear algebra is a fundamental branch of mathematics that deals with vector spaces and linear mappings between them. Among the various concepts studied in linear algebra, linear transformations play a significant role in understanding how mathematical structures change under mapping functions. Linear transformations provide a powerful framework for studying relationships between vector spaces while preserving essential algebraic properties such as vector addition and scalar multiplication.

The study of linear transformations is important because of its wide range of applications in mathematics, physics, computer science, statistics, and engineering. Many real-world problems can be modeled using linear transformations, making them useful in areas such as geometry, data analysis, and scientific computation. The representation of linear transformations using matrices further simplifies calculations and provides an efficient computational approach to solving mathematical problems.

In this assignment, we explore various concepts related to linear transformations. The work includes theoretical definitions, properties, and examples that help illustrate how linear transformations operate. As the study progresses, additional topics and applications will be discussed to provide a comprehensive understanding of the subject.

1 Linear Transformations

1.1 Definition and Mathematical Concept

A linear transformation is a mapping between two vector spaces that preserves the fundamental operations of vector addition and scalar multiplication. Let V and W be vector spaces over the same field F . A function

$$T : V \rightarrow W$$

is called a linear transformation if for all vectors $u, v \in V$ and all scalars $c \in F$, the following properties hold:

$$T(u + v) = T(u) + T(v), \tag{1}$$

$$T(cv) = cT(v). \tag{2}$$

These properties ensure that the structure of the vector space is preserved under transformation.

1.2 Matrix Representation of Linear Transformations

In finite-dimensional vector spaces, linear transformations can be represented using matrices. If $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a linear transformation, then there exists an $m \times n$ matrix A such that

$$T(x) = Ax$$

where x is a column vector in $\mathbb{R}^n$.

If $\{e_1, e_2, \dots, e_n\}$ is the standard basis of $\mathbb{R}^n$, then the matrix representation of T is given by

$$A = [T(e_1) \quad T(e_2) \quad \cdots \quad T(e_n)].$$

1.3 Kernel and Image

Two important concepts associated with linear transformations are kernel and image.

The **kernel** (or null space) of a linear transformation T is defined as

$$\text{Ker}(T) = \{v \in V : T(v) = 0\}.$$

The kernel represents all vectors that are mapped to the zero vector.

The **image** (or range) of T is defined as

$$\text{Im}(T) = \{T(v) : v \in V\}.$$

The dimensions of kernel and image are related by the Rank-Nullity Theorem:

$$\dim(V) = \text{Rank}(T) + \text{Nullity}(T).$$

1.4 Geometrical Interpretation

Geometrically, linear transformations can be viewed as operations that change the position, orientation, or size of geometric objects while preserving straight lines and the origin.

Common geometric transformations include:

- **Scaling:** Changes the size of objects.

$$T(x, y) = (ax, by)$$

- **Rotation in $\mathbb{R}^2$:** Rotates vectors by an angle θ .

$$T(x, y) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- **Reflection:** Reflects vectors across axes or lines. Example reflection across x-axis:

$$T(x, y) = (x, -y)$$

- **Shear Transformation:** Distorts shape while preserving parallelism.

$$T(x, y) = (x + ky, y)$$

Linear transformations always map lines to lines (or points to points) and preserve the origin, which is a key geometric property.

1.5 Properties

Linear transformations satisfy several important algebraic properties which make them useful in mathematical analysis and applications. Let $T : V \rightarrow W$ be a linear transformation between vector spaces V and W over the same field F .

1.5.1 Property 1: Preservation of Zero Vector

Statement:

$$T(0) = 0$$

Proof:

Since $0 = 0 \cdot v$ for any vector $v \in V$, using the property of linearity,

$$T(0) = T(0v) = 0T(v) = 0.$$

Hence, linear transformations always map the zero vector in V to the zero vector in W .

1.5.2 Property 2: Preservation of Linear Combinations

Statement:

For scalars $a_1, a_2, \dots, a_n$ and vectors $v_1, v_2, \dots, v_n \in V$,

$$T\left(\sum_{i=1}^n a_i v_i\right) = \sum_{i=1}^n a_i T(v_i).$$

Proof:

The proof follows from repeated application of the linearity properties.

For $n = 2$,

$$T(a_1 v_1 + a_2 v_2) = T(a_1 v_1) + T(a_2 v_2) = a_1 T(v_1) + a_2 T(v_2).$$

Using mathematical induction, the result holds for any finite number of vectors.

1.5.3 Property 3: Kernel is a Subspace

Statement:

$$\text{Ker}(T) = \{v \in V : T(v) = 0\}$$

is a subspace of V .

Proof:

To prove this, we verify the subspace conditions.

(i) Zero vector belongs to kernel

$$T(0) = 0$$

Thus $0 \in \text{Ker}(T)$.

(ii) Closed under addition

Let $u, v \in \text{Ker}(T)$. Then

$$T(u) = 0 \quad \text{and} \quad T(v) = 0.$$

Now,

$$T(u + v) = T(u) + T(v) = 0 + 0 = 0.$$

Hence $u + v \in \text{Ker}(T)$.

(iii) Closed under scalar multiplication

For scalar c ,

$$T(cu) = cT(u) = c(0) = 0.$$

Thus $cu \in \text{Ker}(T)$.

Hence, kernel is a subspace.

1.5.4 Property 4: Image is a Subspace

Statement:

$$\text{Im}(T) = \{T(v) : v \in V\}$$

is a subspace of W .

Proof:

(i) Zero vector belongs to image

Since

$$T(0) = 0,$$

we have $0 \in \text{Im}(T)$.

(ii) Closed under addition

Let $w_1 = T(v_1)$ and $w_2 = T(v_2)$ be in the image. Then

$$w_1 + w_2 = T(v_1) + T(v_2) = T(v_1 + v_2).$$

Thus the image is closed under addition.

(iii) Closed under scalar multiplication

For scalar c ,

$$cw_1 = cT(v_1) = T(cv_1).$$

Hence image is a subspace.

1.5.5 Property 5: Rank-Nullity Theorem

For finite dimensional vector spaces,

$$\dim(V) = \text{Rank}(T) + \text{Nullity}(T).$$

This theorem relates the dimension of domain space to the dimensions of image and kernel spaces, providing important structural information about linear transformations.

1.6 Zero Linear Transformation

Let V and W be vector spaces over a field F . The **zero linear transformation** $T : V \rightarrow W$ is defined by

$$T(v) = 0_W \quad \text{for all } v \in V$$

where 0_W is the zero vector in W .

The zero mapping $T(v) = 0$ for all $v \in V$ is a linear transformation.

Let $u, v \in V$ and $c \in F$.

First,

$$T(u + v) = 0$$

But

$$T(u) + T(v) = 0 + 0 = 0$$

Thus

$$T(u + v) = T(u) + T(v)$$

Next,

$$T(cu) = 0$$

and

$$cT(u) = c \cdot 0 = 0$$

Thus

$$T(cu) = cT(u)$$

Hence T satisfies the properties of linearity. Therefore it is a linear transformation.

1.7 Identity Linear Transformation

Let V be a vector space. The **identity transformation**

$$I : V \rightarrow V$$

is defined by

$$I(v) = v \quad \text{for all } v \in V$$

The identity mapping $I(v) = v$ is a linear transformation.

Let $u, v \in V$ and $c \in F$.

First,

$$I(u + v) = u + v$$

But

$$I(u) + I(v) = u + v$$

Thus

$$I(u + v) = I(u) + I(v)$$

Next,

$$I(cu) = cu$$

and

$$cI(u) = cu$$

Thus

$$I(cu) = cI(u)$$

Therefore I is linear.

1.8 Matrix Representation of Linear Transformations

Let $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a linear transformation. Then there exists an $m \times n$ matrix A such that for every vector $x \in \mathbb{R}^n$,

$$T(x) = Ax$$

where x is written as a column vector.

Thus, if

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

then

$$Ax = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

and the result is a vector in $\mathbb{R}^m$.

1.8.1 Example

Consider the transformation

$$T(v_1, v_2, v_3) = (v_1 - 2v_2 + v_3, 0, 0).$$

Let

$$x = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}.$$

Then the transformation can be written as a matrix multiplication:

$$T(x) = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}.$$

Multiplying,

$$T(x) = \begin{bmatrix} 1(v_1) + (-2)(v_2) + 1(v_3) \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} v_1 - 2v_2 + v_3 \\ 0 \\ 0 \end{bmatrix}.$$

Thus the matrix

$$A = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

is called the **matrix representation** of the transformation T .
Hence

$$T(x) = Ax.$$

—

1.9 Matrix Transformations are Linear

Let A be an $m \times n$ matrix. The transformation

$$T(x) = Ax$$

from $\mathbb{R}^n$ to $\mathbb{R}^m$ is a linear transformation.

Let $x, y \in \mathbb{R}^n$ and $c \in \mathbb{R}$.

First we check addition:

$$T(x + y) = A(x + y).$$

Using properties of matrix multiplication,

$$A(x + y) = Ax + Ay.$$

Thus

$$T(x + y) = T(x) + T(y).$$

Next we check scalar multiplication:

$$T(cx) = A(cx).$$

Using matrix properties,

$$A(cx) = c(Ax).$$

Thus

$$T(cx) = cT(x).$$

Since both properties hold,

$$T(x + y) = T(x) + T(y), \quad T(cx) = cT(x),$$

the transformation $T(x) = Ax$ is linear.

1.10 Differential Operator

1.10.1 Differential Operator on Functions

Let $C^1(\mathbb{R})$ denote the vector space of all real-valued differentiable functions on $\mathbb{R}$.

Define the mapping

$$D : C^1(\mathbb{R}) \rightarrow C(\mathbb{R})$$

by

$$D(f) = \frac{df}{dx}$$

for every $f \in C^1(\mathbb{R})$. This operator D is called the **differential operator**.

The differential operator D is a linear transformation.

Let $f, g \in C^1(\mathbb{R})$ and $c \in \mathbb{R}$.

First consider addition:

$$D(f + g) = \frac{d}{dx}(f(x) + g(x))$$

Using the derivative rule,

$$= \frac{df}{dx} + \frac{dg}{dx}$$

$$= D(f) + D(g)$$

Next consider scalar multiplication:

$$D(cf) = \frac{d}{dx}(cf(x))$$

$$= c \frac{df}{dx}$$

$$= cD(f)$$

Thus the operator satisfies

$$D(f + g) = D(f) + D(g), \quad D(cf) = cD(f)$$

Hence D is a linear transformation.

If $f(x) = x^3 + 2x^2 + 5$, then

$$D(f) = \frac{d}{dx}(x^3 + 2x^2 + 5) = 3x^2 + 4x.$$

—

1.10.2 Differential Operator on Polynomial Spaces

Let P_n denote the vector space of all polynomials of degree at most n .

Define

$$D : P_n \rightarrow P_{n-1}$$

by

$$D(p(x)) = \frac{dp(x)}{dx}.$$

The differential operator D is a linear transformation from P_n to P_{n-1} .

Let $p(x), q(x) \in P_n$ and $c \in \mathbb{R}$.

First,

$$\begin{aligned} D(p(x) + q(x)) &= \frac{d}{dx}[p(x) + q(x)] \\ &= \frac{dp(x)}{dx} + \frac{dq(x)}{dx} \\ &= D(p(x)) + D(q(x)) \end{aligned}$$

Next,

$$\begin{aligned} D(cp(x)) &= \frac{d}{dx}[cp(x)] \\ &= c \frac{dp(x)}{dx} \\ &= cD(p(x)) \end{aligned}$$

Thus D satisfies the properties of linearity. Hence D is a linear transformation.

Let

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

Then

$$D(p(x)) = a_1 + 2a_2x + 3a_3x^2 + \cdots + na_nx^{n-1}.$$

Thus the degree reduces by one, so $D(p(x)) \in P_{n-1}$.

1.11 Definite Integral as a Linear Transformation

Let $C[a, b]$ denote the vector space of all continuous real-valued functions defined on the closed interval $[a, b]$.

Define the mapping

$$T : C[a, b] \rightarrow \mathbb{R}$$

by

$$T(f) = \int_a^b f(x) dx$$

for every $f \in C[a, b]$.

This mapping T is called the **definite integral transformation**.

The transformation

$$T(f) = \int_a^b f(x) dx$$

is a linear transformation from $C[a, b]$ to $\mathbb{R}$.

Let $f, g \in C[a, b]$ and $c \in \mathbb{R}$.

First we verify additivity.

$$T(f + g) = \int_a^b (f(x) + g(x)) dx$$

Using the linearity property of definite integrals,

$$\begin{aligned} &= \int_a^b f(x) dx + \int_a^b g(x) dx \\ &= T(f) + T(g). \end{aligned}$$

Next we verify scalar multiplication.

$$\begin{aligned} T(cf) &= \int_a^b cf(x) dx \\ &= c \int_a^b f(x) dx \\ &= cT(f). \end{aligned}$$

Since

$$T(f + g) = T(f) + T(g), \quad T(cf) = cT(f),$$

the mapping T satisfies the properties of linearity.

Therefore T is a linear transformation.

Let $f(x) = x$ on the interval $[0, 1]$.

$$T(f) = \int_0^1 x dx$$

$$\begin{aligned}
&= \left[\frac{x^2}{2} \right]_0^1 \\
&= \frac{1}{2}.
\end{aligned}$$

Thus the transformation maps the function $f(x) = x$ to the real number $\frac{1}{2}$.

1.12 Transpose Transformation

Let V be the vector space of all $m \times n$ matrices over $\mathbb{R}$. Define the transformation

$$T : V \rightarrow W$$

by

$$T(A) = A^T$$

where A^T denotes the transpose of the matrix A , obtained by interchanging rows and columns.

If

$$A = (a_{ij}),$$

then

$$A^T = (a_{ji}).$$

Linearity

Let A, B be matrices and c be a scalar.

Addition:

$$T(A + B) = (A + B)^T$$

Using properties of transpose,

$$(A + B)^T = A^T + B^T$$

Thus

$$T(A + B) = T(A) + T(B).$$

Scalar multiplication:

$$T(cA) = (cA)^T$$

$$(cA)^T = cA^T$$

Thus

$$T(cA) = cT(A).$$

Hence, the transpose operation is a linear transformation.

1.13 Zero Transformation

Let V be a vector space of matrices. Define

$$T : V \rightarrow V$$

by

$$T(A) = 0$$

where 0 denotes the zero matrix of appropriate size.

Linearity

Let $A, B \in V$ and c be a scalar.

Addition:

$$T(A + B) = 0$$

and

$$T(A) + T(B) = 0 + 0 = 0.$$

Thus

$$T(A + B) = T(A) + T(B).$$

Scalar multiplication:

$$T(cA) = 0$$

and

$$cT(A) = c \cdot 0 = 0.$$

Thus

$$T(cA) = cT(A).$$

Therefore, the zero transformation is linear.

1.14 Identity Transformation

Let V be a vector space of matrices. Define

$$T : V \rightarrow V$$

by

$$T(A) = A.$$

This transformation leaves every matrix unchanged and is called the identity transformation.

Linearity

Let $A, B \in V$ and c be a scalar.

Addition:

$$T(A + B) = A + B$$

and

$$T(A) + T(B) = A + B.$$

Thus

$$T(A + B) = T(A) + T(B).$$

Scalar multiplication:

$$T(cA) = cA$$

and

$$cT(A) = cA.$$

Thus

$$T(cA) = cT(A).$$

Hence, the identity transformation is linear.

Listing 1: Linear Transformation from R3 to R3

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

# 1. Define the Transformation Matrix
T = np.array([[1, 0, -2],
              [0, 1, 1],
              [0, 0, 0]])

# Define Bases
# Null Space: Vector that maps to [0,0,0]
null_basis = np.array([2, -1, 1])

# Row Space: The rows that span the input (1st and 2nd rows of
# T)
row1, row2 = T[0, :], T[1, :]

# Column Space: The columns that span the output (1st and 2nd
# columns of T)
col1, col2 = T[:, 0], T[:, 1]

print("--- Final Analysis ---")
print(f"Null Space Basis: {null_basis}")
print(f"Rank: 2 | Nullity: 1")
print(f"Column Space Basis:\n{np.column_stack((col1, col2))}")

# --- Plotting ---
fig = plt.figure(figsize=(16, 8))

# Grid for planes
u = np.linspace(-5, 5, 10)
v = np.linspace(-5, 5, 10)
```

```

U, V = np.meshgrid(u, v)

# --- PLOT 1: DOMAIN (R3) ---
ax1 = fig.add_subplot(121, projection='3d')
ax1.set_title("1. DOMAIN ( $R^3$ )\nRow Space (Blue) & Null Space (Red)")

# Row Space Plane:  $X = u \cdot \text{row1} + v \cdot \text{row2}$ 
X_row = U*row1[0] + V*row2[0]
Y_row = U*row1[1] + V*row2[1]
Z_row = U*row1[2] + V*row2[2]
ax1.plot_surface(X_row, Y_row, Z_row, alpha=0.3,
                 color='royalblue')

# Null Space Line:  $t \cdot [2, -1, 1]$ 
t_range = np.linspace(-4, 4, 10)
ax1.plot(t_range*null_basis[0], t_range*null_basis[1],
         t_range*null_basis[2],
         color='red', lw=4, label='Null Space Basis (2,-1,1)')

# --- PLOT 2: RANGE (R3) ---
ax2 = fig.add_subplot(122, projection='3d')
ax2.set_title("2. RANGE ( $R^3$ )\nColumn Space (Green)")

# Column Space Plane:  $X = u \cdot \text{col1} + v \cdot \text{col2}$ 
# This correctly uses indices 0, 1, and 2 for the ROWS of the
# basis columns
X_col = U*col1[0] + V*col2[0]
Y_col = U*col1[1] + V*col2[1]
Z_col = U*col1[2] + V*col2[2]
ax2.plot_surface(X_col, Y_col, Z_col, alpha=0.3,
                 color='limegreen')

# Demonstrate Mapping
# Null Space vector maps to Origin
ax2.scatter(0, 0, 0, color='red', s=200, edgecolors='black',
           label='T(Null Space) = (0,0,0)')

# Row Space vector maps to a vector in Column Space
v_row = row1 + row2
v_out = T @ v_row
ax2.quiver(0, 0, 0, v_out[0], v_out[1], v_out[2],
          color='blue', lw=3, label='T(Row Vector)')

# Formatting

```

```

for ax in [ax1, ax2]:
    ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
    ax.set_xlim([-8, 8]); ax.set_ylim([-8, 8]);
    ax.set_zlim([-8, 8])
    ax.legend()

plt.tight_layout()
plt.show()

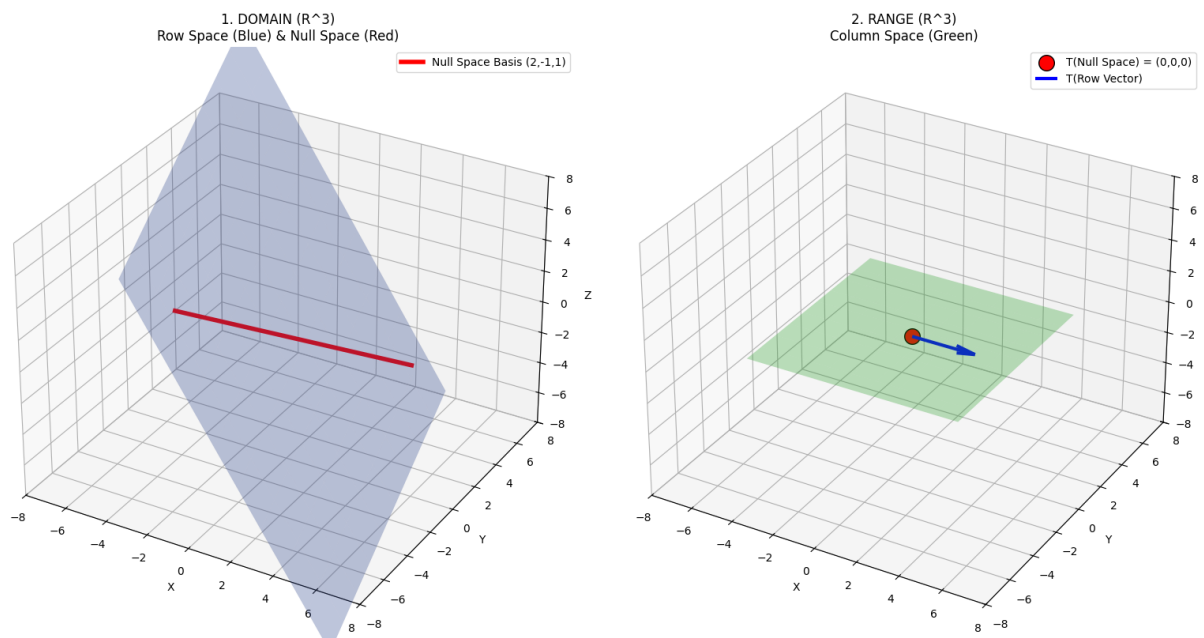
```

OUTPUT

```

--- Final Analysis ---
Null Space Basis: [ 2 -1  1]
Rank: 2 | Nullity: 1
Column Space Basis:
[[1 0]
 [0 1]
 [0 0]]

```



1.15 One-to-One (Injective) Transformations

A linear transformation $T : V \rightarrow W$ is **one-to-one** (or injective) if for all $u, v \in V$, $T(u) = T(v)$ implies $u = v$.

In the context of linear algebra, checking for injectivity is simplified by looking at the kernel:

- T is one-to-one if and only if $\ker(T) = \{\mathbf{0}\}$.

- This implies the **nullity** of the transformation is 0.
- Geometrically, a one-to-one transformation does not "collapse" any non-zero vectors into the origin.

1.16 Onto (Surjective) Transformations

A linear transformation $T : V \rightarrow W$ is **onto** (or surjective) if the range of T is equal to the codomain W , i.e., $\text{im}(T) = W$.

For a transformation to be onto:

- Every vector $w \in W$ must have at least one pre-image $v \in V$ such that $T(v) = w$.
- The **rank** of the transformation must equal the dimension of the codomain ($\text{rank}(T) = \dim(W)$).

1.17 Isomorphisms

A linear transformation $T : V \rightarrow W$ is an **isomorphism** if it is both one-to-one and onto (bijective). If such a transformation exists, V and W are said to be **isomorphic**, denoted $V \cong W$.

Two finite-dimensional vector spaces V and W are isomorphic if and only if $\dim(V) = \dim(W)$.

An isomorphism implies that V and W are structurally identical in terms of linear algebra; they differ only in the "names" of their elements. For an isomorphism represented by a square matrix A , the matrix must be invertible ($\det(A) \neq 0$).

Let $T : V \rightarrow W$ be a linear transformation between finite-dimensional vector spaces V and W . Let $\dim(V) = n$ and $\dim(W) = m$. The fundamental tool for these proofs is the **Rank-Nullity Theorem**:

$$\dim(\ker(T)) + \dim(\text{im}(T)) = \dim(V) \quad (3)$$

Case 1: If $\dim(V) > \dim(W)$, then T is not one-to-one.

A linear transformation T is one-to-one (injective) if and only if $\ker(T) = \{\mathbf{0}\}$, which implies $\dim(\ker(T)) = 0$.

From the Rank-Nullity Theorem:

$$\dim(\ker(T)) = \dim(V) - \dim(\text{im}(T))$$

The image of T is a subspace of W , therefore its dimension cannot exceed the dimension of W :

$$\dim(\text{im}(T)) \leq \dim(W)$$

Substituting this inequality into the Rank-Nullity equation:

$$\dim(\ker(T)) \geq \dim(V) - \dim(W)$$

Given that $\dim(V) > \dim(W)$, it follows that:

$$\dim(V) - \dim(W) > 0 \implies \dim(\ker(T)) > 0$$

Since the dimension of the kernel is at least 1, T has a non-trivial kernel. Thus, T cannot be one-to-one.

Case 2: If $\dim(V) < \dim(W)$, then T is not onto.

A linear transformation T is onto (surjective) if and only if $\text{im}(T) = W$, which implies $\dim(\text{im}(T)) = \dim(W)$.

From the Rank-Nullity Theorem:

$$\dim(\text{im}(T)) = \dim(V) - \dim(\ker(T))$$

Since $\dim(\ker(T)) \geq 0$, the maximum possible value for the dimension of the image is $\dim(V)$:

$$\dim(\text{im}(T)) \leq \dim(V)$$

Given that $\dim(V) < \dim(W)$, we have:

$$\dim(\text{im}(T)) \leq \dim(V) < \dim(W)$$

Since $\dim(\text{im}(T))$ is strictly less than $\dim(W)$, the image of T cannot be the entire space W . Thus, T cannot be onto.

Case 3: If $\dim(V) = \dim(W)$, T is one-to-one $\iff T$ is onto.

Assume $\dim(V) = \dim(W) = k$.

1. **Forward ($\implies$):** Suppose T is one-to-one. Then $\dim(\ker(T)) = 0$. By Rank-Nullity: $0 + \dim(\text{im}(T)) = k$. Thus, $\dim(\text{im}(T)) = k$. Since $\dim(W) = k$ and $\text{im}(T) \subseteq W$, the image must be W . Therefore, T is onto.
2. **Backward ($\impliedby$):** Suppose T is onto. Then $\dim(\text{im}(T)) = \dim(W) = k$. By Rank-Nullity: $\dim(\ker(T)) + k = k$. Subtracting k from both sides gives $\dim(\ker(T)) = 0$. Therefore, T is one-to-one.

The Converse: If T is an Isomorphism, then $\dim(V) = \dim(W)$.

An isomorphism is defined as a linear transformation that is both one-to-one and onto.

- T is one-to-one $\implies \dim(\ker(T)) = 0$.
- T is onto $\implies \dim(\text{im}(T)) = \dim(W)$.

Applying the Rank-Nullity Theorem:

$$\dim(\ker(T)) + \dim(\text{im}(T)) = \dim(V)$$

$$0 + \dim(W) = \dim(V)$$

$$\dim(W) = \dim(V)$$

Thus, the dimensions must be equal for an isomorphism to exist.

2 Applications of Linear Transformations

2.1 2D Reflection Across the y-Axis

A reflection across the y -axis is a linear transformation that maps every point (x, y) to $(-x, y)$. This transformation reverses the horizontal orientation of an object while preserving its vertical position.

Matrix Representation: The transformation matrix M for a reflection across the y -axis is defined as:

$$M = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

Transformation Mapping: Applying the matrix to a general vector $\mathbf{v}$ yields:

$$T(x, y) = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -x \\ y \end{pmatrix}$$

Geometric Interpretation:

- The x -coordinate changes its sign ($x \rightarrow -x$).
- The y -coordinate remains unchanged ($y \rightarrow y$).

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def apply_transformation_2d(matrix, points, title,
description):
    # Apply transformation: T(v) = Av
    transformed_points = matrix @ points

    print("-" * 30)
    print(f"Matrix representation A:\n{matrix}")
    print(f"\nTransformation:\n{description}")
    print("-" * 30)
    print("Point Mapping (First 3 points):")
    for i in range(min(3, points.shape[1])):
        print(f"Original: {points[:, i]} -> Transformed:
            {transformed_points[:, i]}")

    # Plotting
    plt.figure(figsize=(6, 6))
    plt.plot(points[0, :], points[1, :], 'b-o',
        label='Original', alpha=0.3)
    plt.plot(transformed_points[0, :], transformed_points[1,
        :], 'r-o', label='Transformed')
```

```

plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.grid(True, linestyle='--')
plt.legend()
plt.title(title)
plt.axis('equal')
plt.show()

def apply_transformation_3d(matrix, points, title):
    transformed_points = matrix @ points

    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')

    # Original Shape
    ax.scatter(points[0, :], points[1, :], points[2, :],
               c='blue', label='Original')
    # Transformed Shape
    ax.scatter(transformed_points[0, :], transformed_points[1, :],
               transformed_points[2, :], c='red',
               label='Transformed')

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title(title)
    ax.legend()
    plt.show()

# --- 2D Case: Reflection across Y-axis ---
# Define a "House" shape
house_2d = np.array([
    [0, 2, 2, 1, 0, 0], # x-coords
    [0, 0, 2, 3, 2, 0]  # y-coords
])

A_2d = np.array([[-1, 0],
                  [0, 1]])

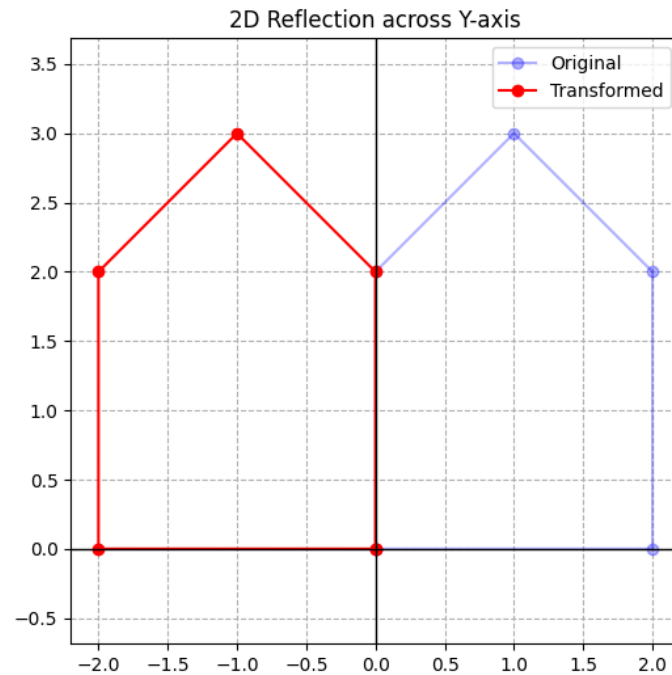
desc_2d = "T(x, y) = (-x, y)\nThe x-coordinate changes\nsign.\nThe y-coordinate remains the same.\nPoints are\nreflected across the y-axis."

apply_transformation_2d(A_2d, house_2d, "2D Reflection across\nY-axis", desc_2d)

```

OUTPUT

```
-----  
Matrix representation A:  
[[-1  0]  
 [ 0  1]]  
  
Transformation:  
T(x, y) = (-x, y)  
The x-coordinate changes sign.  
The y-coordinate remains the same.  
Points are reflected across the y-axis.  
-----  
Point Mapping (First 3 points):  
Original: [0 0] -> Transformed: [0 0]  
Original: [2 0] -> Transformed: [-2 0]  
Original: [2 2] -> Transformed: [-2 2]
```



2.2 Reflection Across the x-Axis

Reflection across the x -axis is a linear transformation that flips a shape vertically. In this mapping, the horizontal positions of the points remain unchanged, while their vertical positions are inverted.

Matrix Representation: The standard matrix A for this transformation is:

$$A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Transformation Rule: The mapping $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is defined by:

$$T(x, y) = (x, -y)$$

Geometric Interpretation:

- The x -coordinate remains the same.
- The y -coordinate changes its sign.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Define the Shape (Vertices of a house)
shape = np.array([
    [0, 2, 2, 1, 0, 0],
    [0, 0, 2, 3, 2, 0]
])

# 2. Define the Transformation Matrix: T(x, y) = (x, -y)
A = np.array([
    [1, 0],
    [0, -1]
])

# 3. Apply Transformation
transformed_shape = A @ shape

# 4. Print Output
print("Matrix representation:\nA =\n", A)
print("\nTransformation: T(x, y) = (x, -y)")
print("Points are reflected across the x-axis.\n")
print("Point Transformations:")
for i in range(shape.shape[1] - 1):
    print(f"P{i+1}: {shape[:, i]} -> {transformed_shape[:, i]}")

# 5. Fixed Plotting Logic
fig, ax = plt.subplots(figsize=(6, 6))
ax.plot(shape[0, :], shape[1, :], 'b-o', label='Original',
        lw=2)
```

```

ax.plot(transformed_shape[0, :], transformed_shape[1, :],
        'r-o', label='Transformed', lw=2)

# Ensure aspect ratio is equal without triggering warnings
ax.set_aspect('equal', adjustable='box')
ax.axhline(0, color='black', lw=1)
ax.axvline(0, color='black', lw=1)
ax.set_xlim(-1, 4)
ax.set_ylim(-4, 4)
ax.grid(True, linestyle='--', alpha=0.6)
ax.legend()
ax.set_title("Reflection across the x-axis")
plt.show()

```

OUTPUT

Matrix representation:

```

A =
[[ 1  0]
 [ 0 -1]]

```

Transformation: $T(x, y) = (x, -y)$

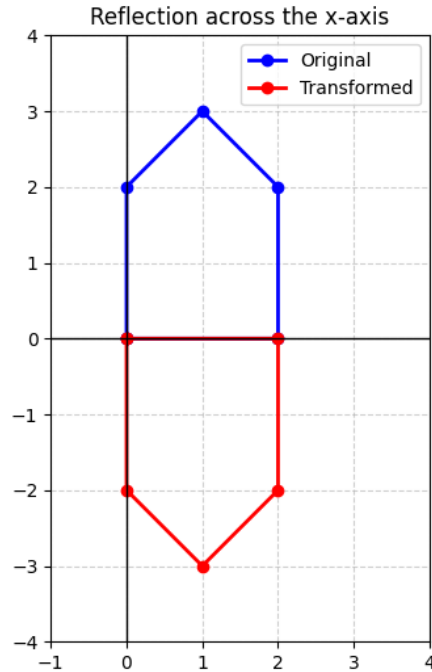
Points are reflected across the x-axis.

Point Transformations:

```

P1: [0 0] -> [0 0]
P2: [2 0] -> [2 0]
P3: [2 2] -> [ 2 -2]
P4: [1 3] -> [ 1 -3]
P5: [0 2] -> [ 0 -2]

```



2.3 Reflection Across the Line $y = x$

This transformation reflects points across the diagonal line where the x and y coordinates are equal. It effectively swaps the horizontal and vertical components of any vector.

Matrix Representation: The standard matrix A for this transformation is:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Transformation Rule: The mapping $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is defined by:

$$T(x, y) = (y, x)$$

Geometric Interpretation:

- The x and y coordinates are interchanged.
- Geometrically, this is a reflection across the 45° line passing through the origin.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Define the Shape
shape = np.array([
    [0, 2, 2, 1, 0, 0],
    [0, 0, 2, 3, 2, 0]
])
```

```

# 2. Define the Transformation Matrix:  $T(x, y) = (y, x)$ 
A = np.array([
    [0, 1],
    [1, 0]
])

# 3. Apply Transformation
transformed_shape = A @ shape

# 4. Print Output
print("Matrix representation:\nA =\n", A)
print("\nTransformation:  $T(x, y) = (y, x)$ ")
print("The x and y coordinates are swapped.\n")
print("Point Transformations:")
for i in range(shape.shape[1] - 1):
    print(f"P{i+1}: {shape[:, i]} -> {transformed_shape[:, i]}")

# 5. Fixed Plotting Logic
fig, ax = plt.subplots(figsize=(6, 6))

# Plot the mirror line  $y = x$ 
line_val = np.linspace(-1, 4, 10)
ax.plot(line_val, line_val, 'g--', alpha=0.5, label='y = x')

ax.plot(shape[0, :], shape[1, :], 'b-o', label='Original',
        lw=2)
ax.plot(transformed_shape[0, :], transformed_shape[1, :],
        'r-o', label='Transformed', lw=2)

ax.set_aspect('equal', adjustable='box')
ax.axhline(0, color='black', lw=1)
ax.axvline(0, color='black', lw=1)
ax.set_xlim(-1, 4)
ax.set_ylim(-1, 4)
ax.grid(True, linestyle='--', alpha=0.6)
ax.legend()
ax.set_title("Reflection across the line  $y = x$ ")
plt.show()

```

OUTPUT

```

Matrix representation:
A =
[[0 1]

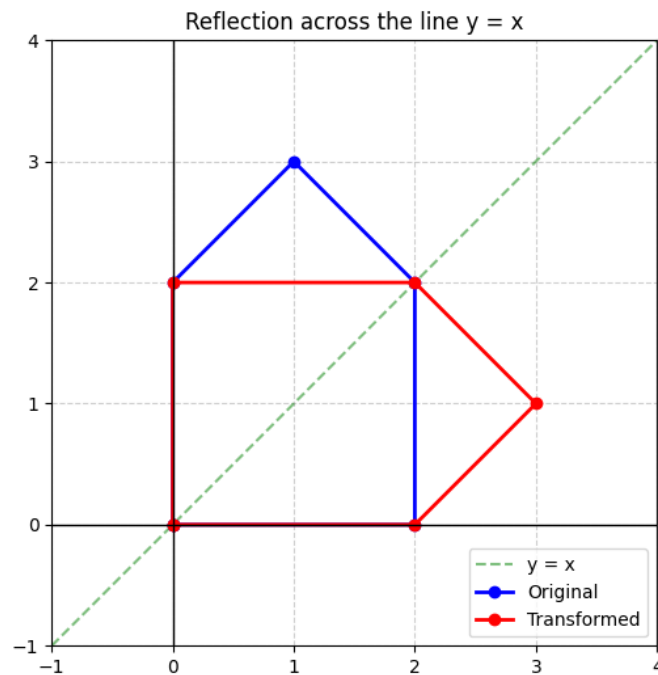
```

```
[1 0]]
```

Transformation: $T(x, y) = (y, x)$
The x and y coordinates are swapped.

Point Transformations:

```
P1: [0 0] -> [0 0]
P2: [2 0] -> [0 2]
P3: [2 2] -> [2 2]
P4: [1 3] -> [3 1]
P5: [0 2] -> [2 0]
```



2.4 Horizontal Scaling

Horizontal scaling stretches or compresses an object along the x -direction. For a scaling factor k , the x -coordinate is multiplied by k while y and z remain constant.

Matrix Representation:

$$A_{2D} = \begin{pmatrix} k & 0 \\ 0 & 1 \end{pmatrix}, \quad A_{3D} = \begin{pmatrix} k & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Transformation Rule: $T(x, y, z) = (kx, y, z)$. With $k = 2$, the object's width is doubled.

```
import numpy as np
```

```

import matplotlib.pyplot as plt

# Transformation Factor
k = 2.0
A_2d = np.array([[k, 0], [0, 1]])
A_3d = np.array([[k, 0, 0], [0, 1, 0], [0, 0, 1]])

# Data
s2d = np.array([[0, 2, 2, 1, 0, 0], [0, 0, 2, 3, 2, 0]])
s3d = np.array([[0,1,1,0,0,1],[0,0,1,1,0,0],[0,0,0,0,1,1]])
edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(0,4),(1,5),(2,5),(3,4)]

print(f"--- Horizontal Scaling (k={k}) ---")
print("2D Matrix:\n", A_2d)
print("\n2D Point Mapping:")
t2 = A_2d @ s2d
for i in range(s2d.shape[1]-1):
    print(f"P{i+1}: ({s2d[0,i]}, {s2d[1,i]}) -> ({t2[0,i]}, {t2[1,i]})")

print("\n3D Matrix:\n", A_3d)
print("\n3D Point Mapping:")
t3 = A_3d @ s3d
for i in range(s3d.shape[1]):
    print(f"V{i+1}: {s3d[:,i]} -> {t3[:,i]}")

# Plotting
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.plot(s2d[0,:], s2d[1,:], 'b--', label='Original')
ax1.plot(t2[0,:], t2[1,:], 'r-o', label='Transformed')
ax1.set_xlabel('X-axis'); ax1.set_ylabel('Y-axis')
ax1.set_aspect('equal', adjustable='box'); ax1.grid(True);
ax1.legend(); ax1.set_title("2D Horizontal Scaling")

ax2 = fig.add_subplot(122, projection='3d')
for s, e in edges:
    ax2.plot(s3d[0,[s,e]], s3d[1,[s,e]], s3d[2,[s,e]], 'b--',
            alpha=0.2)
    ax2.plot(t3[0,[s,e]], t3[1,[s,e]], t3[2,[s,e]], 'r-', lw=2)
ax2.set_xlabel('X'); ax2.set_ylabel('Y'); ax2.set_zlabel('Z')
ax2.set_title("3D Horizontal Scaling")
plt.show()

```

OUTPUT

--- Horizontal Scaling (k=2.0) ---

2D Matrix:

```
[[2. 0.]  
[0. 1.]]
```

2D Point Mapping:

P1: (0, 0) -> (0.0, 0.0)

P2: (2, 0) -> (4.0, 0.0)

P3: (2, 2) -> (4.0, 2.0)

P4: (1, 3) -> (2.0, 3.0)

P5: (0, 2) -> (0.0, 2.0)

3D Matrix:

```
[[2. 0. 0.]  
[0. 1. 0.]  
[0. 0. 1.]]
```

3D Point Mapping:

V1: [0 0 0] -> [0. 0. 0.]

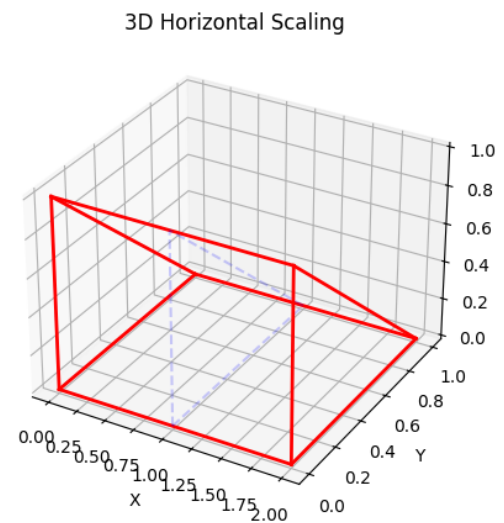
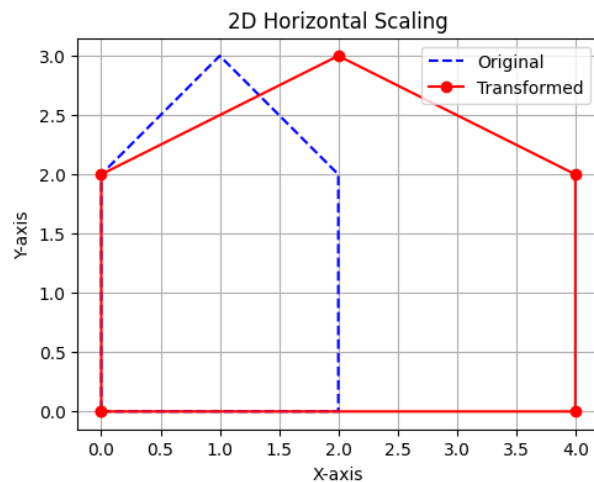
V2: [1 0 0] -> [2. 0. 0.]

V3: [1 1 0] -> [2. 1. 0.]

V4: [0 1 0] -> [0. 1. 0.]

V5: [0 0 1] -> [0. 0. 1.]

V6: [1 0 1] -> [2. 0. 1.]



2.5 Vertical Scaling

Vertical scaling stretches or compresses an object along the y -axis. The height is modified by a factor k , while the width and depth remain constant.

Matrix Representation:

$$A_{2D} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad A_{3D} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
import numpy as np
import matplotlib.pyplot as plt

# Transformation Factor
k = 2.0
A_2d = np.array([[1, 0], [0, k]])
A_3d = np.array([[1, 0, 0], [0, k, 0], [0, 0, 1]])

# Data
s2d = np.array([[0, 2, 2, 1, 0, 0], [0, 0, 2, 3, 2, 0]])
s3d = np.array([[0, 1, 1, 0, 0, 1], [0, 0, 1, 1, 0, 0], [0, 0, 0, 0, 1, 1]])
edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(0,4),(1,5),(2,5),(3,4)]

print(f"--- Vertical Scaling (k={k}) ---")
print("2D Matrix:\n", A_2d)
print("\n2D Point Mapping:")
t2 = A_2d @ s2d
for i in range(s2d.shape[1]-1):
    print(f"P{i+1}: ({s2d[0,i]}, {s2d[1,i]}) -> ({t2[0,i]}, {t2[1,i]})")

print("\n3D Matrix:\n", A_3d)
print("\n3D Point Mapping:")
t3 = A_3d @ s3d
for i in range(s3d.shape[1]):
    print(f"V{i+1}: {s3d[:,i]} -> {t3[:,i]}")

# Plotting
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.plot(s2d[0,:], s2d[1,:], 'b--', label='Original')
ax1.plot(t2[0,:], t2[1,:], 'r-o', label='Transformed')
ax1.set_xlabel('X-axis'); ax1.set_ylabel('Y-axis')
ax1.set_aspect('equal', adjustable='box'); ax1.grid(True);
ax1.legend(); ax1.set_title("2D Vertical Scaling")
```

```

ax2 = fig.add_subplot(122, projection='3d')
for s, e in edges:
    ax2.plot(s3d[0,[s,e]], s3d[1,[s,e]], s3d[2,[s,e]], 'b--',
            alpha=0.2)
    ax2.plot(t3[0,[s,e]], t3[1,[s,e]], t3[2,[s,e]], 'r-', lw=2)
ax2.set_xlabel('X-axis'); ax2.set_ylabel('Y-axis');
    ax2.set_zlabel('Z-axis')
ax2.set_title("3D Vertical Scaling")
plt.show()

```

OUTPUT

```

--- Vertical Scaling (k=2.0) ---

```

```

2D Matrix:

```

```

[[1. 0.]
 [0. 2.]]

```

```

2D Point Mapping:

```

```

P1: (0, 0) -> (0.0, 0.0)
P2: (2, 0) -> (2.0, 0.0)
P3: (2, 2) -> (2.0, 4.0)
P4: (1, 3) -> (1.0, 6.0)
P5: (0, 2) -> (0.0, 4.0)

```

```

3D Matrix:

```

```

[[1. 0. 0.]
 [0. 2. 0.]
 [0. 0. 1.]]

```

```

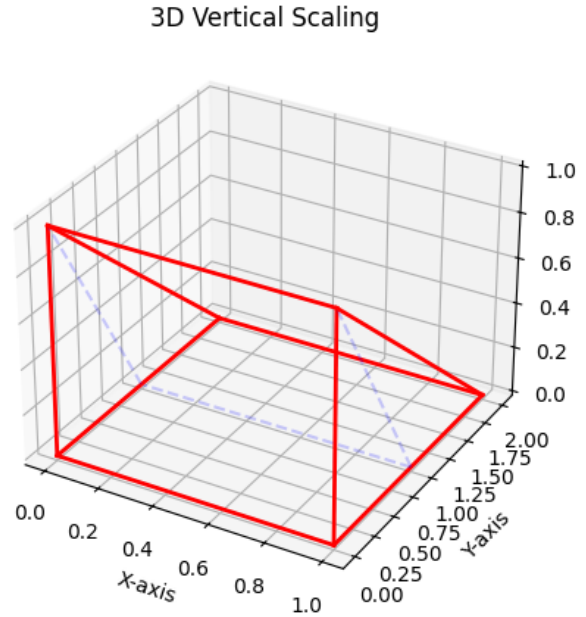
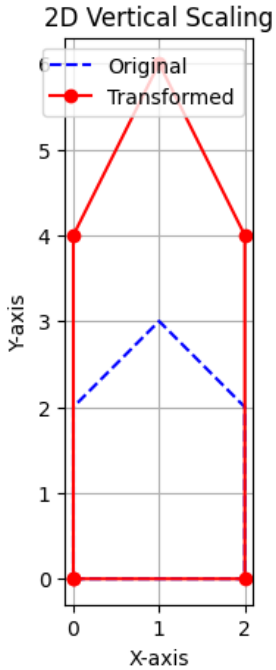
3D Point Mapping:

```

```

V1: [0 0 0] -> [0. 0. 0.]
V2: [1 0 0] -> [1. 0. 0.]
V3: [1 1 0] -> [1. 2. 0.]
V4: [0 1 0] -> [0. 2. 0.]
V5: [0 0 1] -> [0. 0. 1.]
V6: [1 0 1] -> [1. 0. 1.]

```



2.6 Horizontal Shear

A horizontal shear tilts the object along the x -axis. The x -coordinate of each point is shifted by a value proportional to its y -coordinate.

Matrix Representation:

$$A_{2D} = \begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}, \quad A_{3D} = \begin{pmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
import numpy as np
import matplotlib.pyplot as plt

# Transformation Factor
k = 2.0
A_2d = np.array([[1, k], [0, 1]])
A_3d = np.array([[1, k, 0], [0, 1, 0], [0, 0, 1]])

# Data
s2d = np.array([[0, 2, 2, 1, 0, 0], [0, 0, 2, 3, 2, 0]])
s3d = np.array([[0, 1, 1, 0, 0, 1], [0, 0, 1, 1, 0, 0], [0, 0, 0, 0, 1, 1]])
edges = [(0,1), (1,2), (2,3), (3,0), (4,5), (0,4), (1,5), (2,5), (3,4)]

print(f"--- Horizontal Shear (k={k}) ---")
print("2D Matrix:\n", A_2d)
print("\n2D Point Mapping:")
```

```

t2 = A_2d @ s2d
for i in range(s2d.shape[1]-1):
    print(f"P{i+1}: ({s2d[0,i]}, {s2d[1,i]}) -> ({t2[0,i]}, {t2[1,i]})")

print("\n3D Matrix:\n", A_3d)
print("\n3D Point Mapping:")
t3 = A_3d @ s3d
for i in range(s3d.shape[1]):
    print(f"V{i+1}: {s3d[:,i]} -> {t3[:,i]}")

# Plotting
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.plot(s2d[0,:], s2d[1,:], 'b--', label='Original')
ax1.plot(t2[0,:], t2[1,:], 'r-o', label='Transformed')
ax1.set_xlabel('X-axis'); ax1.set_ylabel('Y-axis')
ax1.set_aspect('equal', adjustable='box'); ax1.grid(True);
    ax1.legend(); ax1.set_title("2D Horizontal Shear")

ax2 = fig.add_subplot(122, projection='3d')
for s, e in edges:
    ax2.plot(s3d[0,[s,e]], s3d[1,[s,e]], s3d[2,[s,e]], 'b--',
        alpha=0.2)
    ax2.plot(t3[0,[s,e]], t3[1,[s,e]], t3[2,[s,e]], 'r-', lw=2)
ax2.set_xlabel('X-axis'); ax2.set_ylabel('Y-axis');
    ax2.set_zlabel('Z-axis')
ax2.set_title("3D Horizontal Shear")
plt.show()

```

OUTPUT

```

--- Horizontal Shear (k=2.0) ---
2D Matrix:
[[1. 2.]
 [0. 1.]]

2D Point Mapping:
P1: (0, 0) -> (0.0, 0.0)
P2: (2, 0) -> (2.0, 0.0)
P3: (2, 2) -> (6.0, 2.0)
P4: (1, 3) -> (7.0, 3.0)
P5: (0, 2) -> (4.0, 2.0)

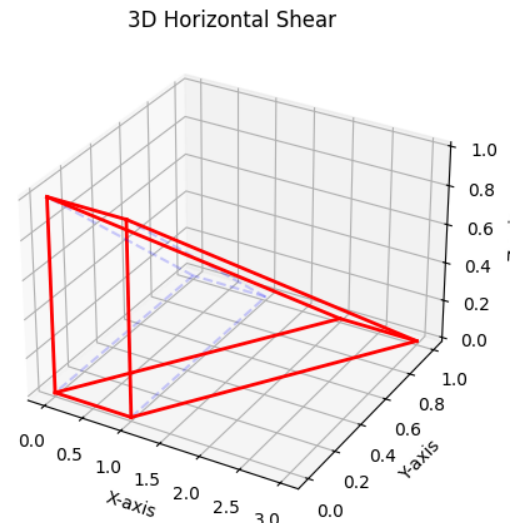
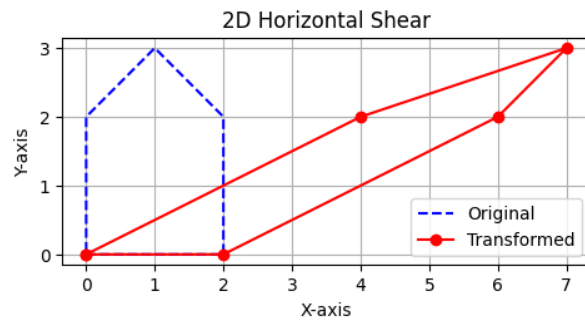
3D Matrix:
[[1. 2. 0.]

```

```
[0. 1. 0.]
[0. 0. 1.]]
```

3D Point Mapping:

```
V1: [0 0 0] -> [0. 0. 0.]
V2: [1 0 0] -> [1. 0. 0.]
V3: [1 1 0] -> [3. 1. 0.]
V4: [0 1 0] -> [2. 1. 0.]
V5: [0 0 1] -> [0. 0. 1.]
V6: [1 0 1] -> [1. 0. 1.]
```



2.7 Vertical Shear

A vertical shear tilts the object along the y -axis. The y -coordinate of each point is shifted by a value proportional to its x -coordinate.

Matrix Representation:

$$A_{2D} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \quad A_{3D} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
import numpy as np
import matplotlib.pyplot as plt

# Transformation Factor
k = 2.0
A_2d = np.array([[1, 0], [k, 1]])
A_3d = np.array([[1, 0, 0], [k, 1, 0], [0, 0, 1]])
```

```

# Data
s2d = np.array([[0, 2, 2, 1, 0, 0], [0, 0, 2, 3, 2, 0]])
s3d = np.array([[0,1,1,0,0,1],[0,0,1,1,0,0],[0,0,0,0,1,1]])
edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(0,4),(1,5),(2,5),(3,4)]

print(f"--- Vertical Shear (k={k}) ---")
print("2D Matrix:\n", A_2d)
print("\n2D Point Mapping:")
t2 = A_2d @ s2d
for i in range(s2d.shape[1]-1):
    print(f"P{i+1}: ({s2d[0,i]}, {s2d[1,i]}) -> ({t2[0,i]}, {t2[1,i]})")

print("\n3D Matrix:\n", A_3d)
print("\n3D Point Mapping:")
t3 = A_3d @ s3d
for i in range(s3d.shape[1]):
    print(f"V{i+1}: {s3d[:,i]} -> {t3[:,i]}")

# Plotting
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.plot(s2d[0,:], s2d[1,:], 'b--', label='Original')
ax1.plot(t2[0,:], t2[1,:], 'r-o', label='Transformed')
ax1.set_xlabel('X-axis'); ax1.set_ylabel('Y-axis')
ax1.set_aspect('equal', adjustable='box'); ax1.grid(True);
ax1.legend(); ax1.set_title("2D Vertical Shear")

ax2 = fig.add_subplot(122, projection='3d')
for s, e in edges:
    ax2.plot(s3d[0,[s,e]], s3d[1,[s,e]], s3d[2,[s,e]], 'b--',
            alpha=0.2)
    ax2.plot(t3[0,[s,e]], t3[1,[s,e]], t3[2,[s,e]], 'r-', lw=2)
ax2.set_xlabel('X-axis'); ax2.set_ylabel('Y-axis');
ax2.set_zlabel('Z-axis')
ax2.set_title("3D Vertical Shear")
plt.show()

```

OUTPUT

```

--- Vertical Shear (k=2.0) ---
2D Matrix:
[[1. 0.]
 [2. 1.]]

2D Point Mapping:

```

```

P1: (0, 0) -> (0.0, 0.0)
P2: (2, 0) -> (2.0, 4.0)
P3: (2, 2) -> (2.0, 6.0)
P4: (1, 3) -> (1.0, 5.0)
P5: (0, 2) -> (0.0, 2.0)

```

3D Matrix:

```

[[1. 0. 0.]
 [2. 1. 0.]
 [0. 0. 1.]]

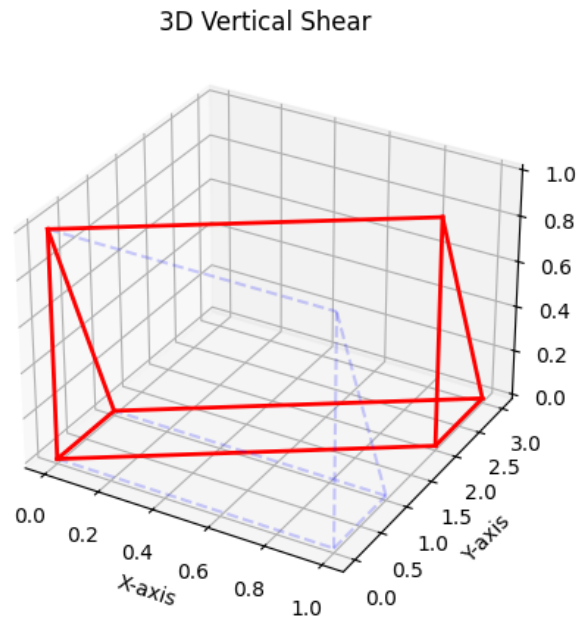
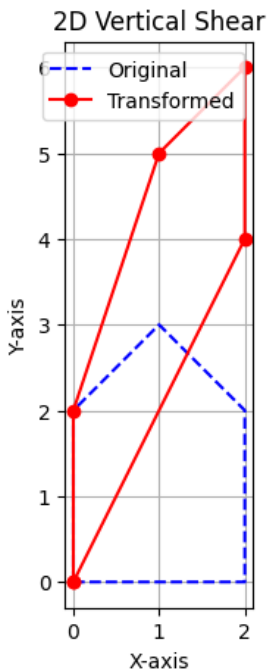
```

3D Point Mapping:

```

V1: [0 0 0] -> [0. 0. 0.]
V2: [1 0 0] -> [1. 2. 0.]
V3: [1 1 0] -> [1. 3. 0.]
V4: [0 1 0] -> [0. 1. 0.]
V5: [0 0 1] -> [0. 0. 1.]
V6: [1 0 1] -> [1. 2. 1.]

```



2.8 Standard Rotation Matrices

For an angle θ , the rotation matrices for the three axes are defined as follows:

Rotation about the x-axis (R_x):

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}$$

Rotation about the z-axis (R_z):

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation about the y-axis (R_y):

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

A common observation in Linear Algebra is that the matrix R_y appears "flipped" compared to R_x and R_z (the negative sine term moves from the top-right to the bottom-left). This is not an error; it is a requirement of the ****Right-Hand Rule**** and ****Cyclic Permutation****.

In a 3D coordinate system, the axes follow a specific sequence:

$$X \rightarrow Y \rightarrow Z \rightarrow X$$

- **X-axis rotation** involves the plane of the next two axes: (y, z) .
- **Z-axis rotation** involves the plane of the next two axes: (x, y) .
- **Y-axis rotation** involves the plane of (z, x) .

Because R_y maps z to x (rather than x to z), the sign of the sine function must be reversed to ensure that a positive θ results in a counter-clockwise rotation when viewed from the positive axis. Without this "flip," the y -axis would follow a Left-Handed orientation, making it inconsistent with the rest of the system.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# 1. Configuration
angle_deg = 45
theta = np.radians(angle_deg)

# 2. X-Axis Rotation Matrix
Rx = np.array([
    [1, 0, 0],
```

```

        [0, np.cos(theta), -np.sin(theta)],
        [0, np.sin(theta),  np.cos(theta)]
    ])

# 3. Define L-Block Vertices (8 points)
# Base part + Chimney part
vertices = np.array([
    [0, 2, 2, 0, 0, 1, 1, 0], # X
    [0, 0, 1, 1, 0, 0, 1, 1], # Y
    [0, 0, 0, 0, 3, 3, 3, 3]  # Z
])

# Define the 6 faces of the L-Block for coloring
def get_faces(v):
    return [
        [v[:,0], v[:,1], v[:,2], v[:,3]], # Bottom
        [v[:,4], v[:,5], v[:,6], v[:,7]], # Top
        [v[:,0], v[:,1], v[:,5], v[:,4]], # Front
        [v[:,2], v[:,3], v[:,7], v[:,6]], # Back
        [v[:,0], v[:,3], v[:,7], v[:,4]], # Left
        [v[:,1], v[:,2], v[:,6], v[:,5]]  # Right
    ]

# 4. Transform Vertices
transformed_vertices = Rx @ vertices

# 5. Console Output (Vertex List)
print(f"--- Rotation about X-Axis (Angle: {angle_deg}°) ---")
print("\nTransformation Matrix (Rx):")
print(np.round(Rx, 3))
print("\nVertex Mapping (Original -> Rotated):")
for i in range(vertices.shape[1]):
    print(f"V{i+1}: {vertices[:,i]} --> {np.round(transformed_vertices[:,i], 2)}")

# 6. Plotting
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Define colors for the faces to make rotation obvious
colors = ['cyan', 'magenta', 'orange', 'gray', 'blue', 'red']

# Plot Original Shape (Transparent/Dashed)
orig_faces = get_faces(vertices)
ax.add_collection3d(Poly3DCollection(orig_faces,

```

```

    facecolors='blue', linewidths=1, edgecolors='blue',
    alpha=0.1))

# Plot Rotated Shape (Solid/Colorful)
rot_faces = get_faces(transformed_vertices)
poly = Poly3DCollection(rot_faces, facecolors=colors,
    linewidths=2, edgecolors='black', alpha=0.8)
ax.add_collection3d(poly)

# Labeling and Formatting
ax.set_xlabel('X-AXIS'); ax.set_ylabel('Y-AXIS');
    ax.set_zlabel('Z-AXIS')
ax.set_title(f"3D Rotation about X-Axis ({angle_deg}°)")

# Set plot limits
ax.set_xlim([-1, 4]); ax.set_ylim([-3, 4]); ax.set_zlim([-1,
    4])

# Add text on the plot for the Matrix and Angle
stats_text = f"Rotation: X-Axis\nAngle: {angle_deg}
    degrees\ndet(R) = {np.linalg.det(Rx):.1f}"
ax.text2D(0.05, 0.95, stats_text, transform=ax.transAxes,
    bbox=dict(facecolor='white', alpha=0.5))

plt.show()

```

OUTPUT

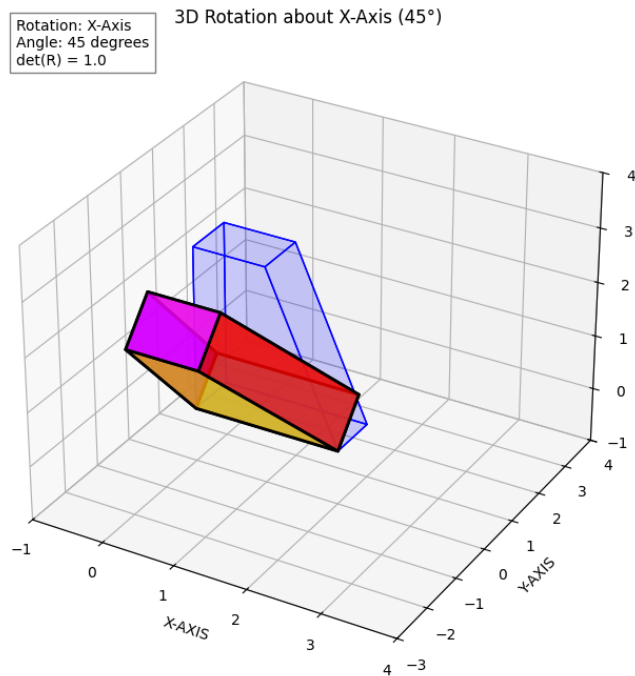
```

--- Rotation about X-Axis (Angle: 45°) ---

Transformation Matrix (Rx):
[[ 1.      0.      0.   ]
 [ 0.      0.707 -0.707]
 [ 0.      0.707  0.707]]

Vertex Mapping (Original -> Rotated):
V1: [0 0 0] ---> [0. 0. 0.]
V2: [2 0 0] ---> [2. 0. 0.]
V3: [2 1 0] ---> [2.  0.71 0.71]
V4: [0 1 0] ---> [0.  0.71 0.71]
V5: [0 0 3] ---> [ 0.  -2.12  2.12]
V6: [1 0 3] ---> [ 1.  -2.12  2.12]
V7: [1 1 3] ---> [ 1.  -1.41  2.83]
V8: [0 1 3] ---> [ 0.  -1.41  2.83]

```



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# 1. Configuration
angle_deg = 45
theta = np.radians(angle_deg)

# 2. Y-Axis Rotation Matrix (Note the placement of sin/cos for Y-axis)
Ry = np.array([
    [np.cos(theta), 0, np.sin(theta)],
    [0, 1, 0],
    [-np.sin(theta), 0, np.cos(theta)]
])

# 3. Define L-Block Vertices (8 points)
vertices = np.array([
    [0, 2, 2, 0, 0, 1, 1, 0], # X
    [0, 0, 1, 1, 0, 0, 1, 1], # Y
    [0, 0, 0, 0, 3, 3, 3, 3]  # Z
])

def get_faces(v):
    return [
```

```

        [v[:,0], v[:,1], v[:,2], v[:,3]], # Bottom
        [v[:,4], v[:,5], v[:,6], v[:,7]], # Top
        [v[:,0], v[:,1], v[:,5], v[:,4]], # Front
        [v[:,2], v[:,3], v[:,7], v[:,6]], # Back
        [v[:,0], v[:,3], v[:,7], v[:,4]], # Left
        [v[:,1], v[:,2], v[:,6], v[:,5]]  # Right
    ]

# 4. Transform Vertices
transformed_vertices = Ry @ vertices

# 5. Console Output
print(f"--- Rotation about Y-Axis (Angle: {angle_deg}°) ---")
print("\nTransformation Matrix (Ry):")
print(np.round(Ry, 3))
print("\nVertex Mapping (Check: Y-values should remain the
      same):")
for i in range(vertices.shape[1]):
    print(f"V{i+1}: {vertices[:,i]} -->
          {np.round(transformed_vertices[:,i], 2)}")

# 6. Plotting
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
colors = ['cyan', 'magenta', 'orange', 'gray', 'blue', 'red']

# Original (Ghost) and Rotated (Solid)
ax.add_collection3d(Poly3DCollection(get_faces(vertices),
    facecolors='blue', alpha=0.05, edgecolors='blue',
    linestyle='--'))
ax.add_collection3d(Poly3DCollection(get_faces(transformed_vertices),
    facecolors=colors, linewidths=2, edgecolors='black',
    alpha=0.8))

ax.set_xlabel('X-AXIS'); ax.set_ylabel('Y-AXIS');
    ax.set_zlabel('Z-AXIS')
ax.set_title(f"3D Rotation: Y-Axis ({angle_deg}°)")
ax.set_xlim([-4, 4]); ax.set_ylim([-1, 4]); ax.set_zlim([-4,
    4])
plt.show()

```

OUTPUT

```

--- Rotation about Y-Axis (Angle: 45°) ---

Transformation Matrix (Ry):

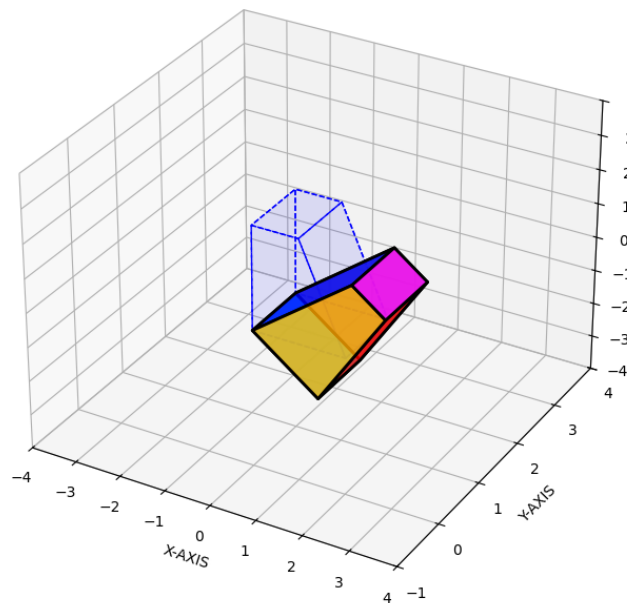
```

```
[[ 0.707  0.      0.707]
 [ 0.      1.      0.   ]
 [-0.707  0.      0.707]]
```

Vertex Mapping (Check: Y-values should remain the same):

```
V1: [0 0 0] ---> [0. 0. 0.]
V2: [2 0 0] ---> [ 1.41  0.   -1.41]
V3: [2 1 0] ---> [ 1.41  1.   -1.41]
V4: [0 1 0] ---> [0.  1.  0.]
V5: [0 0 3] ---> [2.12  0.   2.12]
V6: [1 0 3] ---> [2.83  0.   1.41]
V7: [1 1 3] ---> [2.83  1.   1.41]
V8: [0 1 3] ---> [2.12  1.   2.12]
```

3D Rotation: Y-Axis (45°)



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# 1. Configuration
angle_deg = 45
theta = np.radians(angle_deg)

# 2. Z-Axis Rotation Matrix
Rz = np.array([
    [np.cos(theta), -np.sin(theta), 0],
```

```

        [np.sin(theta),  np.cos(theta), 0],
        [0,               0,             1]
    ])

# 3. Define L-Block Vertices (8 points)
vertices = np.array([
    [0, 2, 2, 0, 0, 1, 1, 0], # X
    [0, 0, 1, 1, 0, 0, 1, 1], # Y
    [0, 0, 0, 0, 3, 3, 3, 3]  # Z
])

# 4. Transform Vertices
transformed_vertices = Rz @ vertices

# 5. Console Output
print(f"--- Rotation about Z-Axis (Angle: {angle_deg}°)--- ")
print("\nTransformation Matrix (Rz):")
print(np.round(Rz, 3))
print("\nVertex Mapping (Check: Z-values should remain the same):")
for i in range(vertices.shape[1]):
    print(f"V{i+1}: {vertices[:,i]} ---> {np.round(transformed_vertices[:,i], 2)}")

# 6. Plotting
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
colors = ['cyan', 'magenta', 'orange', 'gray', 'blue', 'red']

# Faces definition
def get_faces(v):
    return [[v[:,0], v[:,1], v[:,2], v[:,3]], [v[:,4], v[:,5], v[:,6], v[:,7]],
            [v[:,0], v[:,1], v[:,5], v[:,4]], [v[:,2], v[:,3], v[:,7], v[:,6]],
            [v[:,0], v[:,3], v[:,7], v[:,4]], [v[:,1], v[:,2], v[:,6], v[:,5]]]

# Plotting collections
ax.add_collection3d(Poly3DCollection(get_faces(vertices),
    facecolors='blue', alpha=0.05, edgecolors='blue',
    linestyle='--'))
ax.add_collection3d(Poly3DCollection(get_faces(transformed_vertices),
    facecolors=colors, linewidths=2, edgecolors='black',
    alpha=0.8))

```

```

ax.set_xlabel('X-AXIS'); ax.set_ylabel('Y-AXIS');
    ax.set_zlabel('Z-AXIS')
ax.set_title(f"3D Rotation: Z-Axis ({angle_deg}°)")
ax.set_xlim([-4, 4]); ax.set_ylim([-4, 4]); ax.set_zlim([-1,
    4])
plt.show()

```

OUTPUT

```

--- Rotation about Z-Axis (Angle: 45°) ---

```

Transformation Matrix (Rz):

```

[[ 0.707 -0.707  0.   ]
 [ 0.707  0.707  0.   ]
 [ 0.      0.      1.   ]]

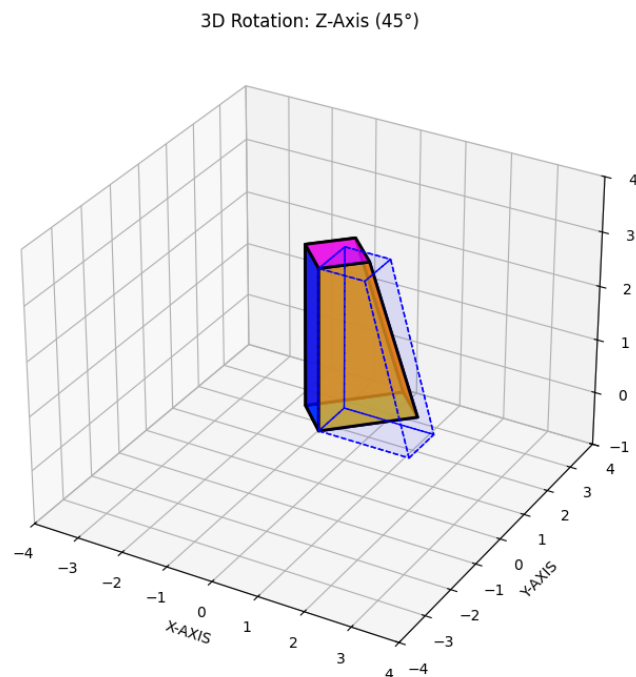
```

Vertex Mapping (Check: Z-values should remain the same):

```

V1: [0 0 0] ---> [0. 0. 0.]
V2: [2 0 0] ---> [1.41 1.41 0.   ]
V3: [2 1 0] ---> [0.71 2.12 0.   ]
V4: [0 1 0] ---> [-0.71 0.71 0.   ]
V5: [0 0 3] ---> [0. 0. 3.]
V6: [1 0 3] ---> [0.71 0.71 3.   ]
V7: [1 1 3] ---> [0.   1.41 3.   ]
V8: [0 1 3] ---> [-0.71 0.71 3.   ]

```



To prove that 3D rotations are non-commutative for any arbitrary angles θ and ϕ , we compare the matrix products $R_y(\theta)R_z(\phi)$ and $R_z(\phi)R_y(\theta)$.

1. Rotation about Y then Z ($R_z(\phi)R_y(\theta)$):

$$R_z(\phi)R_y(\theta) = \begin{pmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

Performing matrix multiplication:

$$= \begin{pmatrix} \cos \phi \cos \theta & -\sin \phi & \cos \phi \sin \theta \\ \sin \phi \cos \theta & \cos \phi & \sin \phi \sin \theta \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

2. Rotation about Z then Y ($R_y(\theta)R_z(\phi)$):

$$R_y(\theta)R_z(\phi) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Performing matrix multiplication:

$$= \begin{pmatrix} \cos \theta \cos \phi & -\cos \theta \sin \phi & \sin \theta \\ \sin \phi & \cos \phi & 0 \\ -\sin \theta \cos \phi & \sin \theta \sin \phi & \cos \theta \end{pmatrix}$$

Analysis: By comparing the two resulting matrices, we observe that:

- Element (2, 1) in the first case is $\sin \phi \cos \theta$, while in the second case it is simply $\sin \phi$.
- Element (2, 3) in the first case is $\sin \phi \sin \theta$, while in the second case it is 0.

Since these terms are not identical for all $\theta, \phi \in \mathbb{R}$, we conclude that $R_z(\phi)R_y(\theta) \neq R_y(\theta)R_z(\phi)$. Thus, the order of rotation is mathematically significant.

While 3D rotations are generally non-commutative, specific angles related to π produce unique mathematical results.

1. Even Multiples of π ($2n\pi = 360^\circ, 720^\circ, \dots$): When a rotation angle is an even multiple of π , the trigonometric values are $\cos(2n\pi) = 1$ and $\sin(2n\pi) = 0$. This reduces the rotation matrix to the **Identity Matrix** (I):

$$R(2n\pi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = I$$

Since the Identity Matrix commutes with all other matrices ($I \cdot A = A \cdot I = A$), rotations by even multiples of π are **commutative**. Physically, this corresponds to a "full circle" rotation that returns the object to its starting orientation.

3 Transition Matrices and Similarity

Let $\mathcal{B}$ and $\mathcal{B}'$ be two bases for a vector space V . We represent these bases as matrices B and B' , where the columns are the basis vectors. Any vector $\mathbf{v}$ can be expressed as:

$$\mathbf{v} = B[\mathbf{v}]_{\mathcal{B}} = B'[\mathbf{v}]_{\mathcal{B}'}$$

3.1 Step 1: The Transition Matrix P

From the equality above, we can isolate the coordinate vector in basis $\mathcal{B}$:

$$[\mathbf{v}]_{\mathcal{B}} = B^{-1}B'[\mathbf{v}]_{\mathcal{B}'}$$

We define the transition matrix P (from $\mathcal{B}'$ to $\mathcal{B}$) as:

$$P = B^{-1}B'$$

Thus, $[\mathbf{v}]_{\mathcal{B}} = P[\mathbf{v}]_{\mathcal{B}'}$.

3.2 Step 2: The Transformation Mapping

Let T be a linear transformation. In basis $\mathcal{B}$, the transformation is represented by matrix A , such that $[T(\mathbf{v})]_{\mathcal{B}} = A[\mathbf{v}]_{\mathcal{B}}$. We wish to find the matrix A' such that $[T(\mathbf{v})]_{\mathcal{B}'} = A'[\mathbf{v}]_{\mathcal{B}'}$.

Starting from the basis equality for the transformed vector $T(\mathbf{v})$:

$$\begin{aligned} B'[T(\mathbf{v})]_{\mathcal{B}'} &= B[T(\mathbf{v})]_{\mathcal{B}} \\ [T(\mathbf{v})]_{\mathcal{B}'} &= (B')^{-1}B[T(\mathbf{v})]_{\mathcal{B}} \end{aligned}$$

3.3 Step 3: Arriving at the Formula

Substitute the relationship $[T(\mathbf{v})]_{\mathcal{B}} = A[\mathbf{v}]_{\mathcal{B}}$ into the equation:

$$[T(\mathbf{v})]_{\mathcal{B}'} = (B')^{-1}B(A[\mathbf{v}]_{\mathcal{B}})$$

Now, substitute $[\mathbf{v}]_{\mathcal{B}} = P[\mathbf{v}]_{\mathcal{B}'}$:

$$[T(\mathbf{v})]_{\mathcal{B}'} = (B')^{-1}BAP[\mathbf{v}]_{\mathcal{B}'}$$

Notice that $(B')^{-1}B$ is the inverse of $P = B^{-1}B'$. Therefore:

$$[T(\mathbf{v})]_{\mathcal{B}'} = P^{-1}AP[\mathbf{v}]_{\mathcal{B}'}$$

Thus, the transformation matrix in the new basis is:

$$A' = P^{-1}AP$$

Two square matrices A and A' are said to be **similar** if there exists an invertible matrix P such that:

$$A' = P^{-1}AP$$

This relationship is an equivalence relation, denoted by $A \sim A'$. In the context of linear transformations, similar matrices represent the same linear operator T expressed in different bases.

Two square matrices A and A' are similar ($A \sim A'$) if there exists an invertible matrix P such that $A' = P^{-1}AP$.

3.4 The Equivalence Relation

Similarity is an equivalence relation, meaning it satisfies reflexivity, symmetry, and transitivity.

1. Reflexivity ($A \sim A$): Let $P = I$. Since the identity matrix is invertible ($I = I^{-1}$):

$$I^{-1}AI = IAI = A$$

Thus, every matrix is similar to itself.

2. Symmetry (If $A \sim A'$, then $A' \sim A$): If $A' = P^{-1}AP$, then multiplying by P on the left and P^{-1} on the right gives:

$$PA'P^{-1} = P(P^{-1}AP)P^{-1} = (PP^{-1})A(PP^{-1}) = IAI = A$$

Let $Q = P^{-1}$. Then $Q^{-1} = P$. Substituting these:

$$Q^{-1}A'Q = A$$

Since Q is invertible, A' is similar to A .

3. Transitivity (If $A \sim B$ and $B \sim C$, then $A \sim C$): Suppose $B = P^{-1}AP$ and $C = Q^{-1}BQ$ for invertible P, Q . Substituting B :

$$C = Q^{-1}(P^{-1}AP)Q = (Q^{-1}P^{-1})A(PQ)$$

Using the property $(PQ)^{-1} = Q^{-1}P^{-1}$, let $M = PQ$:

$$C = M^{-1}AM$$

Since the product of invertible matrices is invertible, A is similar to C .

Let A and A' be square matrices of order n . We say $A \sim A'$ if there exists a transition matrix P such that $A' = P^{-1}AP$.

Requirement: For the inverse P^{-1} to exist, the transition matrix P must be non-singular. This is guaranteed if and only if:

$$\det(P) \neq 0$$

If this condition is met, the following properties remain invariant:

1. Determinant Invariance

The determinant of the similar matrix A' is calculated using the multiplicative property of determinants, $\det(XY) = \det(X)\det(Y)$:

$$\det(A') = \det(P^{-1}AP) = \det(P^{-1})\det(A)\det(P)$$

Since $\det(P^{-1}) = \frac{1}{\det(P)}$, we have:

$$\det(A') = \frac{1}{\det(P)} \cdot \det(A) \cdot \det(P)$$

$$\det(A') = \det(A)$$

Thus, the volume-scaling factor of the transformation is preserved.

2. Trace Invariance

The trace (tr) is the sum of the diagonal elements. The proof utilizes the *cyclic property* of the trace, $\text{tr}(XY) = \text{tr}(YX)$:

$$\text{tr}(A') = \text{tr}(P^{-1}(AP))$$

By commuting P^{-1} to the end of the product:

$$\text{tr}(A') = \text{tr}((AP)P^{-1}) = \text{tr}(A(P P^{-1})) = \text{tr}(AI)$$

$$\text{tr}(A') = \text{tr}(A)$$

3. Characteristic Polynomial and Eigenvalues

Two matrices are similar if they represent the same operator in different bases; therefore, their characteristic equations must be identical:

$$p_{A'}(\lambda) = \det(A' - \lambda I) = \det(P^{-1}AP - \lambda P^{-1}IP)$$

Factoring out the transition matrices:

$$p_{A'}(\lambda) = \det(P^{-1}(A - \lambda I)P) = \det(P^{-1}) \det(A - \lambda I) \det(P)$$

$$p_{A'}(\lambda) = \det(A - \lambda I) = p_A(\lambda)$$

Because the polynomials are identical, the **eigenvalues** (roots of the polynomial) are also identical.

4. Rank Invariance

The rank of a matrix is the dimension of its image. Multiplication by an invertible matrix (which represents an isomorphism) preserves the dimension of the column space:

$$\text{rank}(A') = \text{rank}(P^{-1}AP)$$

Since P and P^{-1} are non-singular ($\det(P) \neq 0$):

$$\text{rank}(P^{-1}AP) = \text{rank}(AP) = \text{rank}(A)$$